

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского
Институт информационных технологий, математики и механики

Отчет по лабораторной работе
«Вычисление арифметических выражений»

Выполнил:
студент группы 3824Б1ПМ4
Овчинников Вячеслав
Александрович

Проверил:
Бусько П.В.

Нижний Новгород
2025

Содержание

Введение	1
Цели и задачи	1
Руководство пользователя.....	1
Руководство программиста.....	1
Результаты тестирования.....	1
Заключение	1
Литература	1
Приложение 1.....	1

Введение

Обратная польская запись, также известная как постфиксная нотация, представляет собой альтернативный способ записи математических выражений. Вместо того чтобы ставить оператор между операндами (как в обычной инфиксной записи), в постфиксной записи оператор располагается после своих операндов. Это позволяет избежать использования скобок для определения порядка выполнения операций, поскольку порядок действий становится очевидным из самой структуры записи. Вычисление таких выражений становится более прямолинейным, особенно при использовании стека.

Стек — это особая структура данных, которая работает по принципу "последним вошел — первым вышел" (LIFO). Представьте себе стопку тарелок: вы можете добавить новую тарелку только сверху, и снять тоже можно только верхнюю. В контексте вычислений стек идеально подходит для временного хранения данных. Он используется для накопления промежуточных результатов вычислений и операторов, когда мы преобразуем обычные (инфиксные) выражения в постфиксные, а затем и для самого процесса вычисления итогового значения.

В этой лабораторной работе мы реализовали два основных компонента: шаблонный класс `Stack`, который предоставляет функциональность для работы со стеком (добавление, удаление, проверка пустоты и т.д.), и класс `TPostfix`. Класс `TPostfix` отвечает за преобразование арифметических выражений из инфиксной записи в постфиксную, а также за их последующее вычисление. Он поддерживает стандартные арифметические операции и некоторые математические функции.

Цели и задачи

Цель работы: Исследование и практическая реализация структуры данных "Стек" с последующим применением ее для решения задачи вычисления арифметических выражений, используя алгоритмы преобразования в обратную польскую запись.

Задачи:

- **Разработка абстракции стека:** Создать шаблонный класс Stack, инкапсулирующий логику стека и предоставляющий следующие интерфейсы: push (вставка), pop (извлечение), top (просмотр), isEmpty (проверка на пустоту), size (получение количества элементов), clear (очистка). Реализовать динамическое управление памятью.
- **Создание парсера и вычислителя выражений:** Разработать класс TPostfix, отвечающий за:
 - Трансляцию арифметических выражений из инфиксной формы в постфиксную.
 - Выполнение вычислений над выражениями в постфиксной записи.
 - Поддержку полного набора арифметических операторов (+, -, *, /, ^) и распространенных математических функций (sin, cos, ln, exp).
 - Валидацию корректности синтаксиса выражений.
- **Обеспечение качества кода:** Разработать набор модульных тестов с использованием Google Test для верификации правильности работы каждого элемента системы.

Руководство пользователя

Основная концепция программы — перевод выражение из инфиксной записи в обратную польскую запись.

```
=== ARITHMETIC EXPRESSION CALCULATOR ===

Supported operations:
- Basic: +, -, *, /, ^ (power)
- Functions: sin(x), cos(x), ln(x), exp(x)
- Parentheses: ( )
- Numbers: integers and decimals (using dot)

IMPORTANT: Arguments for sin and cos are in degrees!
Expression examples:
  2 + 3 * 4
  (1 + 2) * 3
  sin(30) + cos(60)
  2^3 + ln(exp(5))

=====
Enter an expression (or 'exit' or 'q' to quit):
```

рис.1 - Пользовательское

Оно показывает пользователю, какие операции и функции определены, как правильно писать пример и как выйти из программы.

При правильном вводе оно показывает инфиксную форму, обратную польскую запись и сам результат.

```
12+3

RESULTS:
  Original expression: 12+3
  Postfix notation: 12 3 +
  Calculation result: 15

=====
Enter an expression (or 'exit' or 'q' to quit):
```

рис.2 - Пользовательское меню. Вывод.

Руководство программиста

Описание программной реализации класса Stack:

Класс Stack реализован как шаблонный класс, что позволяет создавать стеки для различных типов данных.

Приватные поля:

T* mem — указатель на динамический массив для хранения элементов

int size — текущая емкость стека

int top — индекс верхнего элемента

Основные методы:

Конструкторы:

```
public:
    Stack(): mem(new T[8]), size(8), top(0) {};    // создает стек с начальной емкостью 8
    explicit Stack(int sz) : size(sz), top(0) {} //создает стек заданной емкости
    if (sz <= 0) throw std::invalid_argument("the stack size must be positive");
    mem = new T[sz];
}
```

рис.3 - Конструкторы.

Деструктор: автоматически освобождает выделенную память.

Метод resize(): автоматически увеличивает емкость стека при переполнении:

```
void resize() {
    size_t newsize = (size == 0) ? 8 : size * 2;
    T* newmem = new T[newsize];
    for (size_t i = 0; i < size; ++i) {
        newmem[i] = mem[i];
    }
    delete[] mem;
    mem = newmem;
    size = newsize;
}
```

рис.4 - метод resize().

Основные операции:

- Push(const T& val): Инициировать добавление элемента val в стек.
- T Pop(): Удалить и вернуть верхний элемент из стека.
- T Top(): Предоставить доступ к верхнему элементу стека без его удаления.
- bool isEmpty(): Определить, содержит ли стек какие-либо элементы.
- bool isFull(): Проверить, достигнут ли стек максимальной вместимости.
- int Getsize(): Получить текущее количество элементов, хранящихся в стеке.
- void Clear(): Удалить все элементы из стека, возвращая его в исходное пустое состояние.

1. Описание программной реализации класса TPostfix

Класс TPostfix служит для обработки арифметических выражений, предоставляя функционал для их преобразования и представления в постфиксной нотации.

Приватные члены данных:

- std::string infix: Хранит исходное выражение в инфиксной форме.
- std::string postfix: Содержит эквивалентное выражение, преобразованное в постфиксную форму.

Функция ToPostfix() выполняет преобразование математических выражений из привычной инфиксной формы в постфиксную (обратную польскую нотацию), следуя логике знаменитого алгоритма "Сортировочной станции" Эдгера Дейкстры.

Процесс разворачивается следующим образом:

1. Подготовка: Мы начинаем с пустого стека для временного хранения операторов и пустой строки, куда будет записываться результат.
2. Пошаговый анализ: Затем мы проходим по входному выражению символ за символом:
 - а. Пропускаем любые пробелы.
 - б. Если видим число (будь то целое или с десятичной точкой), сразу добавляем его к результату.
 - в. Распознаем и обрабатываем математические функции, такие как $\sin$, $\cos$, $\ln$, $\exp$.
 - г. Управляем порядком операций с помощью скобок, помещая их в стек или извлекая из него.
 - д. Обрабатываем операторы (+, -, *, / и т.д.), учитывая их установленные приоритеты.
3. Финализация: После того как все символы входной строки обработаны, любые операторы, оставшиеся в стеке, переносятся в окончательную выходную строку.

```
while (i < infix.length()) {  
    char c = infix[i];  
  
    if (isdigit(c) || c == '.') {  
        // Обработка чисел  
    }  
    else if (isalpha(c)) {  
        // Обработка функций  
    }  
    else if (c == '(') {  
        op.Push("(");  
    }  
    else if (c == ')') {  
        // Извлечение операторов до открывающей скобки  
    }  
    else if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {  
        // Обработка операторов с учетом приоритетов  
    }  
}
```

рис.5 - Ключевые части алгоритма

Функция `calculated()` реализует алгоритм вычисления постфиксных выражений. Для этого она использует структуру данных "стек", которая действует по принципу "последним пришел – первым ушел".

Процесс начинается с создания пустого стека. Затем входная строка, представляющая постфиксное выражение, поэтапно разбирается на составные части – токены. Каждый токен обрабатывается следующим образом:

- Если токен является числом, оно помещается на вершину стека.

- Если токен является оператором (например, бинарным), из стека извлекаются два верхних элемента (операнды). Над ними выполняется соответствующая операция, и полученный результат помещается обратно в стек.
- Если токен представляет собой функцию (например, унарную), из стека извлекается один операнд. К нему применяется указанная функция, и результат также сохраняется в стеке.

После того как все токены выражения будут обработаны, в стеке останется ровно одно значение – это и будет окончательный результат вычисления всего выражения.

Поддерживаемые операции и функции

Вы можете использовать следующие математические операции и функции:

Арифметические:

- + Сложение (приоритет 1)
- - Вычитание (приоритет 1)
- * Умножение (приоритет 2)
- / Деление (приоритет 2)
- ^ Возведение в степень (приоритет 4)

Математические:

- $\sin(x)$ Синус (x в градусах)
- $\cos(x)$ Косинус (x в градусах)
- $\ln(x)$ Натуральный логарифм
- $\exp(x)$ Экспонента

Результаты тестирования

Проверка класса Stack

Созданы модульные тесты, охватывающих следующие аспекты:

Конструкторы:

- Инициализация стека по умолчанию
- Создание стека с заданным размером
- Обработка некорректных значений размера

Основные операции:

- Правильность добавления и удаления элементов
- Корректная работа методов Top(), GetSize(), isEmpty()
- Очистка содержимого стека

Обработка ошибок:

- Попытка извлечения элемента из пустого стека
- Получение верхнего элемента при пустом стеке

Перевыделение памяти:

- Автоматическое расширение емкости стека
- Сохранение данных при увеличении размера

Итог: Все тесты выполнены успешно.

Проверка класса TPostfix

Разработаны модульные тесты, проверяющих:

Преобразование выражений:

- Простые арифметические операции
- Выражения со скобками
- Выражения с использованием функций
- Сложные составные выражения

Вычисление значений:

- Корректность вычислений
- Правильное соблюдение приоритетов операций
- Работа с математическими функциями

Обработка ошибок:

- Непарные скобки
- Недопустимые символы в выражениях
- Деление на ноль
- Некорректный формат чисел
- Неизвестные функции

Особые случаи:

- Выражения с пробелами
- Пустые выражения
- Выражения с отрицательными числами

Итог: Все тесты пройдены успешно, программа корректно обрабатывает разнообразные типы выражений и ошибки.

Заключение

В процессе выполнения лабораторной работы были успешно выполнены все поставленные задачи.

Был создан шаблонный класс Stack, представляющий структуру данных «стек» и включающий полный набор необходимых операций. Этот класс обеспечивает:

- Эффективное управление памятью,
- Автоматическое расширение вместимости при переполнении,
- Правильную обработку пограничных случаев,
- Типобезопасность за счёт использования шаблонов.

Также разработан класс TPostfix для работы с арифметическими выражениями, который:

- Преобразует выражения из инфиксной записи в постфиксную форму,
- Выполняет вычисления с учётом приоритетов операций,
- Поддерживает основные математические функции,
- Осуществляет тщательную проверку корректности выражений.

Создана автоматизированная система тестирования на базе Google Test, включающая 44 модульных тестов, охватывающих:

- Основные варианты использования,
- Пограничные ситуации,
- Обработку ошибок.

Доказана практическая полезность стека как структуры данных для:

- Управления приоритетом операций,
- Хранения промежуточных результатов вычислений,
- Обработки скобок в выражениях.

Ключевые результаты работы:

- Полная реализация алгоритма преобразования выражений,
- Корректная обработка ошибок с указанием их положения в строке,
- Поддержка математических функций и операции возведения в степень,
- Эффективное применение стека для вычислений,
- Высокое качество кода с соблюдением принципов объектно-ориентированного программирования.

Разработанная программа демонстрирует стабильное выполнение операций с различными типами выражений и может служить основой для создания более сложных систем обработки математических выражений.

Литература

1. **Никлаус Вирт.** Алгоритмы и структуры данных. / Пер. с англ. Ткачев Ф. В. — М.: ДМК Пресс, 2010. — 272 с.
2. **Страуструп, Бьерн.** Язык программирования C++ [Текст] / Бьерн Страуструп ; пер. с англ. под ред. Н. Н. Мартынова. - Спец. изд. - Москва : Бином, 2011. - 1135 с. : ил.; 24 см.; ISBN 978-5-7989-0425-9
3. **Язык программирования C#** [Текст] / А. Хейлсберг [и др.] ; [пер. с англ. Р. Тетерин]. - 4-е изд. - Москва [и др.] : Питер, 2012. - 773 с. : ил.; 24 см. - (Классика Computer Science).; ISBN 978-5-459-00283-6
4. **Сластикова, М. Д.** Алгоритмы и структуры данных [Электронный ресурс] : учебное пособие по курсу "Алгоритмы и структуры данных" для студентов направления "Программная инженерия" 231000.62 : электронное издание локального распространения / М. Д. Сластикова, Т. Э. Шульга, О. А. Торопова ; М-во образования и науки Российской Федерации, Саратовский гос. технический ун-т им. Гагарина Ю. А. - Саратов : СГТУ им. Гагарина Ю. А., 2015. - 1 электрон. опт. диск (CD-ROM); 12 см.; ISBN 987-5-7433-2860-4

Приложение 1.

```
void TPostfix::ToPostfix() {
    Stack<string> op;
    stringstream out;
    size_t i = 0;
    int pos = 1;

    while (i < infix.length()) {
        char c = infix[i];

        if (isspace(c)) { i++; pos++; continue; }

        if (isdigit(c) || c == '.') {
            string num = "";
            size_t startpos = pos;

            while (i < infix.length() && (isdigit(infix[i]) || infix[i] == '.')) {
                num += infix[i++];
                pos++;
            }

            if (num.empty() || num == ".") {
                throw runtime_error("error in format number in pos " + to_string(startpos));
            }

            if (num.find(".") != string::npos) {
                throw runtime_error("error in format number in pos " + to_string(startpos));
            }
            try {
                stod(num);
            }
            catch (...) {
                throw runtime_error("error in format number in pos " + to_string(startpos));
            }

            out << num << " ";
            continue;
        }

        if (isalpha(c)) {
            string name;
            while (i < infix.length() && isalpha(infix[i])) {
                name += infix[i];
                i++;
                pos++;
            }
            pos += name.length();

            if (!func.count(name)) throw runtime_error("error in format func");
        }
    }
}
```

```

if (i >= infix.length() || infix[i] != '(') throw runtime_error("after func need '('");

op.Push(name);
op.Push("(");
i++; pos++;
continue;

}

if (c == '(') {
op.Push("(");
i++; pos++;
continue;
}

if (c == ')') {
while (!op.isEmpty() && op.Top() != "(") out << op.Pop() << " ";

if (op.isEmpty()) throw runtime_error("extra ')' " + to_string(pos));

op.Pop();

if (!op.isEmpty() && func.count(op.Top())) out << op.Pop() << " ";

i++; pos++;
continue;
}
if(c=='+'||c=='-'||c=='*'||c=='/' || c=='^'){
string op_str(1, c);
int prio = prior[op_str];
bool right_assoc = (c == '^');

while (!op.isEmpty() && op.Top() != "(" && (prior[op.Top()] > prio || (prior[op.Top()] == prio
&& !right_assoc))) {
out << op.Pop() << " ";
}
op.Push(op_str);
i++; pos++;
continue;
}

throw runtime_error(" nedopustim simvol " + string(1, c) + "(pos " + to_string(pos) + ")");
}

while (!op.isEmpty()) {
if (op.Top() == "(") throw runtime_error("ne hvataet ");
out << op.Pop() << " ";
}

postfix = out.str();

```

}

Приложение 1 - программная реализация метода
ToPostfix().